

AASTool Complete Project Documentation

AASTool — Complete Project Documentation

Enterprise Edition | Version 1.0.0 | July 2026

1. Executive Summary

What Is AASTool?

AASTool is a web-based software system that evaluates how accessible buildings are for people with disabilities. Think of it like a **building accessibility report card** — it measures a building against European standards (EN 17210) and gives it a score from 0 to 100, plus a letter grade from A+ (excellent) down to D (poor).

Why Does It Exist?

Buildings need to be accessible to everyone. Architects, building owners, and government inspectors need a standardised way to measure and certify accessibility. AASTool provides that standard.

Who Uses It?

- **Architects** — to check their designs meet accessibility standards
- **Building owners** — to certify their buildings
- **Accessibility auditors** — to perform professional evaluations
- **Government agencies** — to enforce accessibility regulations
- **Consultants** — to generate certification reports

Business Goal

To provide an enterprise-grade, data-driven certification platform that produces professional PDF reports aligned with European standards.

2. System Overview

High-Level Architecture

WEB BROWSER
(React / Next.js)

User interface that
displays data and
collects scores

HTTP REST API

BACKEND API
(Express / Node.js)

All business logic,
calculations, and
data processing

Database queries

DATABASE
(MariaDB)

All reference data,
criteria, weights,
saved assessments

Three Main Components

Component	Technology	Purpose
Frontend	Next.js 16 + React 19	User interface — what users see and click
Backend	Express 4 + TypeScript	Brain of the system — all calculations
Database	MariaDB 10.11	Memory of the system — stores all data

Communication Model

- The frontend **never** talks to the database
- The frontend **only** talks to the backend via HTTP REST API
- The backend **never** serves web pages — only JSON data
- The database is the **single source of truth** for all data

3. Project Structure (Full Tree)

/proj/

```

backend/                                # Express API server
  src/
    index.ts                            # Entry point - starts the server
    data-source.ts                      # Database connection configuration
    controllers/                        # 6 files - handle HTTP requests
      authController.ts
      assessmentsController.ts
      buildingsController.ts
      dataController.ts
      evaluateController.ts
      reportsController.ts
    entities/                          # 7 files - database table definitions
      Criterion.ts
      BuildingType.ts
      Evaluation.ts
      NebThreshold.ts
      Config.ts
      DisabilityType.ts
      AssessmentDimension.ts
    services/                          # 2 files - business logic
      calculationService.ts            # Scoring engine
      metadataService.ts               # Data access layer
    scripts/                          # Utility scripts
      seed.ts                         # Populates database with data
      runMigrations.ts                # Runs database schema changes
    data/                             # Static reference data
      criteria.ts                     # 63 evaluation criteria
      building-types.json             # Building profiles with weights
    migrations/                       # Database schema versioning
    tests/                            # 4 test files
  Dockerfile                          # Container build instructions
  package.json                        # Dependencies and scripts

frontend/                             # Next.js web application
  app/                               # Pages (13 routes)
    page.tsx                         # Home / landing page
    layout.tsx                       # Root layout (header, nav, footer)
    globals.css                      # Global styles
    dashboard/                      # Main dashboard with KPIs
    certification/                  # Scoring and certification engine
    criteria/                       # Criteria reference table
    buildings/                      # Building list and detail
    disability/                    # Disability types overview
    dimensions/                    # Assessment dimensions overview
    reports/                       # Generated reports list
    login/                         # Login page
    assessments/                   # Assessment creation and detail

```

components/	# Reusable UI components
Header.tsx	# Top navigation bar
BottomNav.tsx	# Mobile navigation
SplashScreen.tsx	# Loading animation
ErrorBoundary.tsx	# Error handling
layout/	# Navbar and theme provider
lib/	# Shared utilities
api/	# API client (Axios) and endpoints
hooks/	# React hooks for data fetching
pdfExport.ts	# PDF report generation
theme.ts	# Colors and gradients
storage.ts	# Local storage helpers
animations.ts	# Animation presets
types/	# TypeScript type definitions
database/	# Standalone database project
docker-compose.yml	# MariaDB + Adminer containers
docker/initdb/	# First-run SQL schema
migrations/	# Migration files (copied from backend)
scripts/	# Backup, restore, schema export
docs/	# Database architecture documentation
verify-database.sh	# Data integrity checker
azure/	# Azure deployment templates
main.bicep	# Subscription-level deployment
resources.bicep	# Resource group configuration
docs/	# Documentation
api.md	# API reference
backend.md	# Backend architecture
frontend.md	# Frontend architecture
azure-deployment.md	# Azure deployment guide
deploy-backend-database.md	
workflow.md	# Development workflow
scripts/	# Development scripts
start-dev.sh	# Start all services
stop-dev.sh	# Stop all services
start-api.sh	# Start API only
status-dev.sh	# Check service status
build-all.sh	# Build and start production
check-deploy.sh	# Pre-deployment validation
.github/workflows/deploy.yml	# CI/CD pipeline

4. Frontend Explanation

Technology Stack

Technology	Version	Purpose
Next.js	16.2.6	Web framework — pages and routing
React	19.2.6	User interface components
TypeScript	5.9	Type safety (prevents bugs)
Tailwind CSS	4	Styling — makes things look good
TanStack React Query	5	Data fetching and caching
Axios	1.5	HTTP client — talks to the API
Framer Motion	12.4	Animations
jsPDF	4.2	PDF generation

How Every Folder Works

app/ — Pages

Each folder inside `app/` is a route (webpage):

Route	File	What It Does
/	<code>page.tsx</code>	Landing page with logo and links
/dashboard	<code>dashboard/page.tsx</code>	Main dashboard showing KPIs, bar charts
/certification	<code>certification/page.tsx</code>	Score page — score criteria, export PDF
/criteria	<code>criteria/page.tsx</code>	Browse all 63 criteria with search/filter
/buildings	<code>buildings/page.tsx</code>	List of building types
/buildings/[id]	<code>buildings/[id]/page.tsx</code>	Single building detail
/disability	<code>disability/page.tsx</code>	Information about 5 disability types
/dimensions	<code>dimensions/page.tsx</code>	Information about 5 assessment dimensions
/reports	<code>reports/page.tsx</code>	List of generated assessment reports
/login	<code>login/page.tsx</code>	Login form
/assessments/new	<code>assessments/new/page.tsx</code>	Create new assessment
/assessments/[id]	<code>assessments/[id]/page.tsx</code>	View assessment result

components/ — Shared UI Parts

Component	Purpose
<code>Header.tsx</code>	Top bar with logo and theme toggle
<code>BottomNav.tsx</code>	Bottom navigation on mobile
<code>SplashScreen.tsx</code>	Animated loading screen
<code>ErrorBoundary.tsx</code>	Catches errors — prevents blank screen
<code>BuildingCard.tsx</code>	Card showing building summary
<code>OfflineIndicator.tsx</code>	Banner when internet is lost
<code>PWAProvider.tsx</code>	Install-as-app prompt
<code>layout/mobile-nav.tsx</code>	Mobile nav + dark/light theme
<code>layout/navbar.tsx</code>	Desktop sidebar navigation

lib/ — Shared Code

File	Purpose
api/client.ts	Axios setup — configures how API calls are made
api/endpoints.ts	All API call functions (13 functions)
hooks/useBuildings.ts	Fetches building data
hooks/useCriteria.ts	Fetches criteria data
hooks/useAssessment.ts	Fetches assessment by ID
hooks/useMetadata.ts	Fetches DT/AD labels
pdfExport.ts	Generates the 9-page PDF report
theme.ts	Color definitions and gradients
storage.ts	Saves/loads scores from browser storage
animations.ts	Animation presets
queryClient.ts	React Query configuration
pwa.ts	Service worker registration

Data Flow in the Frontend

User clicks button

Component calls endpoint function (e.g., `endpoints.evaluate()`)

Axios sends HTTP request to backend API

Backend returns JSON response

React Query caches the result

Component re-renders with new data

User sees updated UI

API Client (`api/client.ts`)

The frontend uses **Axios** to make HTTP calls:

```
const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL, // Configurable!
  timeout: 15000, // 15 seconds
});
```

Key rule: **The API base URL is set via environment variable.** Change one variable to point the frontend to any backend.

PDF Export (lib/pdfExport.ts)

When the user clicks “Export PDF” on the certification page, the frontend:

1. Collects all scores and results from the current evaluation
2. Generates a 9-page PDF using `jspdf` library
3. Downloads it automatically

The PDF includes: cover page, table of contents, executive summary, methodology, DT analysis, AD analysis, strengths/weaknesses, criteria details, and recommendations.

5. Backend Explanation

Technology Stack

Technology	Version	Purpose
Node.js	22	JavaScript runtime
Express	4.18	Web server framework
TypeScript	5.9	Type safety
TypeORM	0.3.17	Database ORM (object-relational mapper)
MariaDB	10.11	Database driver
Helmet	7	Security headers
Compression	1.7	Response compression (faster loads)

How Every File Works

Entry Point: `src/index.ts`

This is where the server starts. It:

1. Loads environment variables
2. Configures security (Helmet, CORS, rate limiting)
3. Connects to the database
4. Registers all API routes
5. Starts listening on port 4000
6. Handles graceful shutdown (cleanly closes connections)

Controllers — Handle HTTP requests:

Controller	File	Endpoints	Purpose
Auth	<code>authController.ts</code>	<code>POST /login</code>	User authentication
Evaluate	<code>evaluateController.ts</code>	<code>POST /evaluate</code>	Run scoring engine
Data	<code>dataController.ts</code>	<code>GET</code> endpoints	Fetch reference data
Buildings	<code>buildingsController.ts</code>	<code>GET</code> endpoints	List building types
Assessments	<code>assessmentsController.ts</code>	<code>POST</code> endpoints	Create/view assessments
Reports	<code>reportsController.ts</code>	<code>GET /reports</code>	List generated reports

Services — Business Logic:

Service	File	Purpose
Calculation	<code>calculationService.ts</code>	The brain — computes scores
Metadata	<code>metadataService.ts</code>	Reads data from database

Entities — Database Table Definitions:

Entity	Table	What It Stores
Criterion	<code>criteria</code>	63 evaluation questions
BuildingType	<code>building_types</code>	7 building profiles with weights
Evaluation	<code>evaluations</code>	Saved assessment results
NebThreshold	<code>neb_thresholds</code>	Score-to-grade mapping
Config	<code>config</code>	System settings
DisabilityType	<code>disability_types</code>	DT labels and colors
AssessmentDimension	<code>assessment_dimensions</code>	AD labels and colors

Request Flow

HTTP Request arrives

Express middleware (security, JSON parsing, rate limiting)

Route matched (e.g., POST `/api/v1/evaluate`)

Controller validates input

Service executes business logic

Repository queries database (via TypeORM)

JSON response returned to client

All 17 API Endpoints

Method	Endpoint	Controller	Purpose
GET	<code>/health</code>	Inline	Liveness probe (lightweight)

Method	Endpoint	Controller	Purpose
GET	/ready	Inline	Readiness probe (checks database)
GET	/api/v1/health	Inline	Legacy health check
POST	/api/v1/evaluate	evaluateController	Run accessibility evaluation
GET	/api/v1/criteria	dataController	List all 63 criteria
POST	/api/v1/criteria	dataController	Create new criterion
GET	/api/v1/criteria/:code	dataController	Get single criterion
GET	/api/v1/building-types	dataController	List 7 building types
GET	/api/v1/neb-thresholds	dataController	Get NEB grade thresholds
GET	/api/v1/config	dataController	Get system configuration
GET	/api/v1/disability-types	dataController	Get 5 disability types
GET	/api/v1/assessment-dimensions	dataController	Get 5 assessment dimensions
GET	/api/v1/buildings	buildingsController	List buildings + assessments
GET	/api/v1/buildings/:id	buildingsController	Get building detail
POST	/api/v1/assessments	assessmentsController	Save assessment
GET	/api/v1/assessments/:id	assessmentsController	Get assessment
GET	/api/v1/reports	reportsController	List reports
POST	/api/v1/login	authController	Login

Response Format

Every endpoint returns consistent JSON:

```
// Success
{ "ok": true, "data": { ... } }
```

```
// Error
{ "ok": false, "error": "Human-readable message" }
```

HTTP status codes: 200 (success), 201 (created), 400 (bad request), 404 (not found), 500 (server error).

6. Database Explanation

What Is Stored?

The database has 7 tables:

Table	Records	What It Stores
criteria	63	Evaluation questions (e.g., “Is the entrance wide enough?”)
building_types	7	Building profiles with importance weights
evaluations	Varies	Saved assessment results
neb_thresholds	5	Score-to-grade mapping
config	5	System settings
disability_types	5	Disability category labels
assessment_dimensions	5	Assessment area labels

The 5 Disability Types (DT)

ID	Name	Example
1	Physical Disability	Wheelchair users, mobility impairments
2	Sensory Disability	Blind, deaf users
3	Cognitive & Neurodiverse	Memory, attention conditions
4	Communication & Mental Health	Speech, anxiety
5	Multiple / Situational	Combined conditions

The 5 Assessment Dimensions (AD)

ID	Name	What It Measures
1	Spatial & Physical	Ramps, doors, corridors
2	Safety & Environmental	Lighting, acoustics, hazards
3	Cognitive & Navigational	Signage, wayfinding
4	Digital & Smart	Kiosks, apps, smart systems
5	Social Inclusion	Dignity, privacy, participation

Scoring Formula

Each criterion is scored 1–5:

Impact Score (IS) = Criterion Weight × User Score

Example: - Criterion “Entrance Width” has weight 0.7 - User scores it 4 out of 5 - Impact Score = $0.7 \times 4 = 2.8$

NEB Classification

Score	Grade	Meaning
85	A+	Excellent
60	A	Good
40	B	Acceptable
20	C	Below Average

Score	Grade	Meaning
0	No Rating	Non-compliant

7. Data Flow — Complete Example

Let's trace what happens when a user evaluates a building:

Step 1: User Opens Certification Page

Browser → `http://localhost:3000/certification`

Frontend fetches criteria from API: `GET /api/v1/criteria`

Frontend fetches building types: `GET /api/v1/building-types`

Page renders with all 63 criteria, each showing a score selector (1-5)

Step 2: User Assigns Scores

User changes score for "Entrance Width" from 3 to 5

Frontend saves score to browser storage (`localStorage`)

UI updates to show new score visually

Step 3: User Clicks “Evaluate”

Frontend sends: `POST /api/v1/evaluate`

Body: `{ "buildingType": "Commercial Buildings", "scores": { "EC1.1.1": 5, ... } }`

Backend receives request

Controller validates input

Calculation service runs:

1. Loads building type weights from database
2. Loads all 63 criteria from database
3. For each criterion: $IS = \text{weight} \times \text{user score (or default 3)}$

4. Groups results by disability type → TIS scores
5. Groups results by dimension → CIS scores
6. Calculates Overall Building Score (OBS) 0-100
7. Determines NEB class from thresholds
8. Identifies strengths (score 5) and weaknesses (score 2)

Returns JSON: { "ok": true, "result": { "obs": 72.45, "nebClass": "A", ... } }

Step 4: Frontend Displays Results

Frontend receives evaluation result

Updates dashboard with:

- OBS score gauge
- NEB class badge
- Score distribution chart
- Strengths and weaknesses lists
- Per-disability-type averages
- Per-dimension averages

Step 5: User Exports PDF

User clicks "Export PDF"

Frontend generates 9-page PDF using jsPDF:

- Page 1: Cover page with score and grade
- Page 2: Table of contents
- Page 3: Executive summary with KPI cards
- Page 4: Methodology explanation
- Pages 5-6: DT and AD analysis
- Page 7: Strengths and weaknesses
- Page 8: Full criteria table
- Page 9: Recommendations

PDF downloads automatically

8. Business Logic Explained Simply

The Scoring System

Think of a building accessibility inspection like a school report card:

- **63 questions** (criteria) covering every aspect of a building
- Each question scored **1 to 5** (1 = poor, 5 = excellent)

- Each question has a **weight** (some things matter more than others)
- Questions are grouped by **who benefits** (disability type) and **what's measured** (dimension)

How the Score Is Calculated

1. **Per question:** Multiply weight $\times$ score
2. **Per disability type:** Add up all scores for questions that matter to that group
3. **Per dimension:** Add up all scores for questions in that building area
4. **Overall:** Combine everything into a percentage (0-100)

The Grade (NEB)

The percentage is converted to a grade: - **A+ (85+)**: Excellent — best practice - **A (60-84)**: Good - **B (40-59)**: Acceptable - **C (20-39)**: Below Average - **No Rating (0-19)**: Non-compliant

Strengths and Weaknesses

- **Strength:** Any question scored 5/5 — these are highlighted as achievements
 - **Weakness:** Any question scored 1-2/5 — these are flagged for improvement
-

9. Deployment Guide

Local Development

```
# 1. Start database
cd database
docker compose up -d

# 2. Start backend
cd backend
npm install
npm run build
export DATABASE_URL="mysql://myuser:mypassword@127.0.0.1:3306/mydb"
npm start

# 3. Start frontend
cd frontend
npm install
export NEXT_PUBLIC_API_URL="http://localhost:4000/api/v1"
npm run dev
```

Production (Azure)

The backend is deployed to Azure App Service. The database is a managed MariaDB service. The frontend is deployed separately to Azure Static Web Apps.

Key environment variables for Azure:

Variable	Example	Purpose
DATABASE_URL	mysql://user:pass@host:3306/database	Database connection
NODE_ENV	production	Production mode
CORS_ORIGIN	https://app.azurestaticapps.net	Allowed frontend URL
JWT_SECRET	Random 64-char string	Authentication key

Docker

The backend has a Dockerfile that creates a small, secure container:

Multi-stage build:

1. Builder stage - compiles TypeScript
2. Production stage - minimal image with only what's needed
 - Non-root user for security
 - Health check (every 30 seconds)
 - Only production dependencies

10. Security Overview

Layer	Protection	How
HTTP	Helmet headers	Prevents common web attacks
API	CORS restriction	Only approved domains can call API
API	Rate limiting	Max 100 requests per 15 minutes
API	Request size limit	Max 1MB per request
Database	Parameterized queries	Prevents SQL injection (via TypeORM)
Input	JSON validation	Rejects malformed input
Auth	HMAC-SHA256 tokens	Password-based authentication
Secrets	Environment variables	No credentials in code

Data Protection

- No personal data stored by default
- All database queries use TypeORM (parameterized — safe from injection)
- Environment variables never committed to version control
- `.env` files are excluded via `.gitignore`

11. Troubleshooting

Problem	Likely Cause	Solution
Frontend shows blank page	API not running	Start backend with <code>npm start</code>
“Cannot connect to database”	MariaDB not running	<code>docker compose up -d in database/</code>
“CORS error” in browser	Wrong frontend URL	Set <code>CORS_ORIGIN</code> environment variable
PDF export does nothing	Popup blocker	Allow popups for the site
Login returns error	<code>JWT_SECRET</code> not set	Add <code>JWT_SECRET</code> to <code>.env</code> file
Scores not saving	Browser storage full	Clear browser cache and reload
Build fails	Node version mismatch	Use Node.js 22 (<code>node --version</code>)

12. Appendix — Developer Notes

Naming Conventions

- **Files:** `camelCase.ts` for utilities, `PascalCase.tsx` for components
- **API endpoints:** lowercase-with-hyphens (e.g., `/api/v1/building-types`)
- **Database tables:** `snake_case` (e.g., `building_types`)
- **TypeScript:** `camelCase` for variables, `PascalCase` for types

Architecture Rules

1. **Frontend NEVER accesses the database**
2. **Backend NEVER serves HTML** — only JSON
3. **All business logic lives in the backend**
4. **All weights and thresholds live in the database**
5. **Every API response follows the same format:** `{ ok, data/error }`
6. **No hardcoded URLs** — everything is configurable via environment variables

Testing Strategy

Test Level	Location	What It Tests
Static analysis	<code>tests/static.test.ts</code>	No hardcoded values in code
Data integrity	<code>tests/workbook.mapping.test.ts</code>	All criteria are valid
Calculation	<code>tests/calculation.test.ts</code>	Scoring produces correct results
Metadata	<code>tests/metadata.test.ts</code>	Database data is complete
API smoke	<code>check-deploy.sh</code>	All endpoints respond correctly

Future Improvements

1. **Real user authentication** — database-backed users with roles
2. **PDF customization** — company logos, custom colors
3. **Multiple building comparison** — side-by-side score comparison
4. **API pagination** — for large data sets
5. **WebSocket notifications** — real-time updates
6. **Mobile apps** — React Native or PWA enhancements

Scalability

- The backend is **stateless** — can run multiple instances behind a load balancer
- Database connection pooling is handled by TypeORM
- The frontend can be served from a CDN (Azure Static Web Apps)
- Docker containers can be orchestrated with Kubernetes or Azure Container Apps

AASTool by Serg / Dev by Y